

GROUPE 11 : RAPPORT D'ANALYSE DE PERFORMANCE

Dans le cadre de l'optimisation de notre site web d'annonces immobilières, une analyse des performances des requêtes a été réalisée sur la collection « **annonces** ». L'objectif est de démontrer l'efficacité et l'impact de notre stratégie d'indexation face aux requêtes de recherche multi-critères.

A. Analyse "Avant" : COLLSCAN

Nous avons testé la requête suivante, qui représente l'un des filtres les plus fréquents de notre interface utilisateur (filtrage par type et par prix) :

```
db["annonces"].find({
  type: "location",
  prix: { $lt: 500000 }
}).explain('executionStats')
```

Dans cette configuration, aucune indexation n'est définie sur les critères de recherche. Par conséquent, le moteur de base de données est contraint d'effectuer un balayage complet de la collection **annonces** pour filtrer les documents pertinents comme nous pouvons le constater sur les captures d'écrans ci-dessous.

```
>_MONGOOSH
> use immoLib
< switched to db immoLib
> db["annonces"].find({ type: "location", prix: { $lt: 500000 } }).explain('executionStats')
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'immoLib.annonces',
    parsedQuery: {
      '$and': [
        {
          type: {
            '$eq': 'location'
          }
        },
        {
          prix: {
            '$lt': 500000
          }
        }
      ]
    },
    indexFilterSet: false,
    queryHash: 'C05C7C69',
    planCacheShapeHash: 'C05C7C69',
    planCacheKey: 'CB6D4E2D',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
```

```
>_MONGOSH
{
  executionStats: {
    executionSuccess: true,
    nReturned: 26,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 52,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        '$and': [
          {
            type: {
              '$eq': 'location'
            }
          },
          {
            prix: {
              '$lt': 500000
            }
          }
        ]
      },
      nReturned: 26,
      executionTimeMillisEstimate: 0,
      works: 53,
      advanced: 26,
      needTime: 26,
```

```
needTime: 26,
needYield: 0,
saveState: 0,
restoreState: 0,
isEOF: 1,
direction: 'forward',
docsExamined: 52
}
},
queryShapeHash: '94E9767F54A5508250E833ECCCE0AC8C1C4DC011C48F1DFD46392EBAA863E4B3',
command: {
  find: 'annonces',
  filter: {
    type: 'location',
    prix: {
      '$lt': 500000
    }
  },
},
'$db': 'immoLib'
},
```

L'exécution de la requête montre les indicateurs suivants :

- Stage d'exécution : COLLSCAN (Collection Scan)
- Documents examinés (**totalDocsExamined**) : 52
- Résultats retournés (**nReturned**) : 26

Interprétation : Lors de cette opération, le moteur a procédé à un balayage linéaire complet de la collection **annonces**. Pour identifier les 26 documents correspondant aux critères de filtrage, le système a été contraint d'examiner l'intégralité des 52 documents stockés.

Ce comportement, propre au COLLSCAN, démontre une complexité algorithmique linéaire ($O(n)$) qui induit une charge inutile sur les ressources système (I/O et CPU) et limite la scalabilité de l'application à mesure que le volume de données augmente.

B. Analyse "Après" : IXSCAN

Nous avons créé un **index composé** sur les champs type et prix. Cette opération structure les données sous forme d'arbre, permettant au moteur de recherche de localiser les documents instantanément.

Nous avons ensuite réexécuté la même requête pour mesurer le gain d'efficacité :

```
>_MONGOSH
> use immoLib
< switched to db immoLib
> db["annonces"].createIndex({ type: 1, prix: 1 })
< type_1_prix_1
> db["annonces"].find({ type: "location", prix: { $lt: 500000 } }).explain('executionStats')
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'immoLib.annonces',
    parsedQuery: {
      '$and': [
        {
          type: {
            '$eq': 'location'
          }
        },
        {
          prix: {
            '$lt': 500000
          }
        }
      ]
    }
  },
}
```

Grâce à cet index, le moteur n'a plus besoin de parcourir toute la collection ; il accède directement aux résultats, comme le démontrent les captures ci-dessous :

```
>_MONGOSH
executionStats: {
  executionSuccess: true,
  nReturned: 26,
  executionTimeMillis: 0,
  totalKeysExamined: 26,
  totalDocsExamined: 26,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 26,
    executionTimeMillisEstimate: 0,
    works: 27,
    advanced: 26,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 26,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 26,
      executionTimeMillisEstimate: 0,
      works: 27,
      advanced: 26,
      needTime: 0,
      needYield: 0,
```

```
>_MONGOSH
keypattern: {
  type: 1,
  prix: 1
},
indexName: 'type_1_prix_1',
isMultiKey: false,
multiKeyPaths: {
  type: [],
  prix: []
},
isUnique: false,
isSparse: false,
isPartial: false,
indexVersion: 2,
direction: 'forward',
indexBounds: {
  type: [
    '["location", "location"]'
  ],
  prix: [
    '[-inf.0, 500000)'
  ]
},
keysExamined: 26,
seeks: 1,
dupsTested: 0,
dupsDropped: 0
}
}
```

L'exécution de la requête montre les indicateurs suivants :

- Stage d'exécution : `IXSCAN`
- Documents examinés (**totalDocsExamined**) : 26
- Résultats retournés (**nReturned**) : 26

Interprétation : L'indexation a transformé la méthode d'accès aux données. Le moteur utilise désormais une structure en arbre B-Tree qui permet de pointer directement vers les adresses physiques des documents ciblés.

Le nombre de documents examinés est devenu strictement égal au nombre de résultats retournés. L'efficacité est optimale : le moteur ne traite plus aucune donnée inutile, réduisant ainsi la charge CPU et I/O de manière drastique.

En conclusion, cette comparaison prouve que l'indexation est essentielle pour optimiser notre site web. Le passage d'un balayage complet (COLLSCAN) à une recherche indexée (IXSCAN) réduit de moitié le travail du serveur, passant de 52 à 26 documents analysés. Cette méthode permet un accès direct aux données, ce qui garantit la rapidité et la scalabilité du projet, même avec un volume important d'annonces.